HuskyPark Predictor: AI-Driven Parking Spot Probability Dashboard for SCSU

Database Design Report: Phase 2

Course	CSCI 414/514: Platform Based Development
Section	Group 8
Members	Chidubem Okoye, Yvonne Onmakpo, Tonika Devi Avanigadda
Date	March 2026
Institution	St. Cloud State University

Abstract

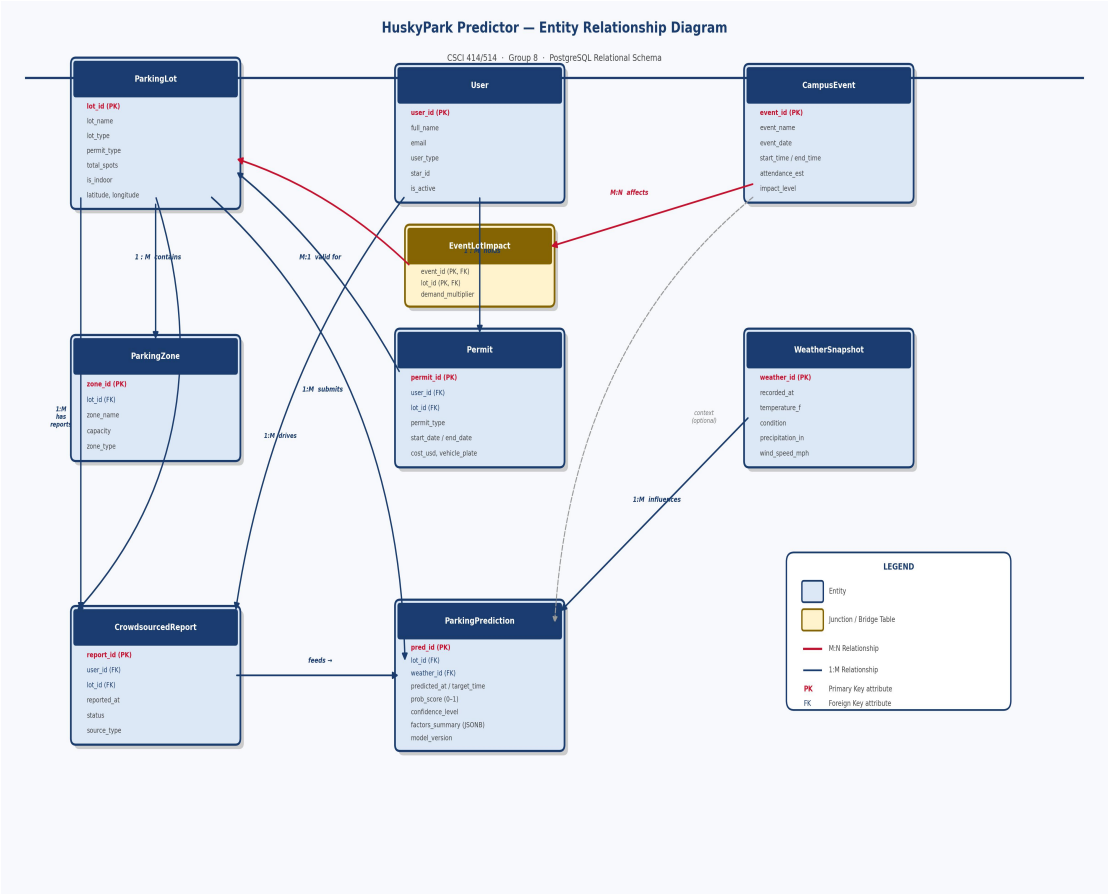
This report presents the complete database architecture for the HuskyPark Predictor which is a web based platform predicting parking availability across Saint Cloud State University campus lots. A PostgreSQL relational database handles structured, strongly consistent operational data, while MongoDB stores flexible analytics, prediction explanations, and user feedback. This revision incorporates four improvements:

1. JSONB typing for structured factor data in ParkingPrediction
2. A clarified SQL versus MongoDB data responsibility boundary that eliminates duplication
3. A graphical ER diagram
4. A realistic SCSU seed data.

B. Relational Database Design

The relational layer uses PostgreSQL for all structured, strongly consistent operational data: lot definitions, user accounts, permits, campus events, weather snapshots, raw crowd reports, and computed parking predictions. This data requires ACID guarantees, complex joins, and enforced constraints, characteristics where a relational model excels.

B.1 Entity Relationship (ER) Model



B.1.1 Entities & Key Relationships

- **ParkingLot:** Core lot registry. Stores name, type, total capacity, permit category, and GPS coordinates for map rendering.
- **ParkingZone:** Sub areas within a lot (e.g., accessible row, EV bays). Enables finer grained capacity tracking.
- **User:** Any person using the system; student, faculty, staff, or visitor. Anchors reports and permits to a person.
- **Permit:** A permit issued to a specific user for a specific lot. Tracks type, dates, cost, and vehicle plate.
- **CrowdsourcedReport:** A raw spot status submission from a user (found_spot / lot_full / hard_to_find). Stored in SQL only to maintain referential integrity and support aggregation queries. The full enriched version is stored in MongoDB.
- **CampusEvent:** Scheduled events (games, open houses, graduations) with estimated attendance and impact level.
- **EventLotImpact:** M:N junction table linking events to the lots they affect, with a demand multiplier per lot.
- **WeatherSnapshot:** Hourly weather observations correlated with prediction scoring.
- **ParkingPrediction:** Computed probability score for a lot at a future time. Includes a JSONB column for structured factor data.

B.2 Entity Attribute Definitions

ParkingLot

Column	Type	Constraints	Description
lot_id	SERIAL	PRIMARY KEY	Auto incrementing lot identifier
lot_name	VARCHAR(50)	NOT NULL, UNIQUE	Human readable name (e.g., 'Lot C', '4th Ave Ramp')
lot_type	VARCHAR(20)	NOT NULL, CHECK	resident/commuter/employee/pay
permit_type	VARCHAR(30)	NOT NULL	Required permit code (e.g., RES, A, AA, VISITOR)
total_spots	INTEGER	NOT NULL, CHECK > 0	Total physical spaces
is_indoor	BOOLEAN	DEFAULT FALSE	TRUE only for 4th Ave Ramp
latitude	DECIMAL(9, 6)	NOT NULL	GPS latitude for map rendering
longitude	DECIMAL(9, 6)	NOT NULL	GPS longitude for map rendering
description	TEXT		Optional notes

ParkingZone

Column	Type	Constraints	Description
zone_id	SERIAL	PRIMARY KEY	Unique zone identifier
lot_id	INTEGER	FK → ParkingLot	Parent lot
zone_name	VARCHAR(40)	NOT NULL	Zone label (e.g., 'Row A', 'Accessible')
capacity	INTEGER	NOT NULL	Number of spaces in this zone
zone_type	VARCHAR(20)	DEFAULT 'Standard'	Standard/Accessible/Reserved/EV

User

Column	Type	Constraints	Description
user_id	SERIAL	PRIMARY KEY	Unique user identifier
full_name	VARCHAR(100)	NOT NULL	Full name
email	VARCHAR(150)	NOT NULL, UNIQUE	SCSU or personal email
user_type	VARCHAR(20)	NOT NULL, CHECK	student/faculty/ staff / visitor
star_id	VARCHAR(20)	UNIQUE	SCSU StarID for campus auth
created_at	TIMESTAMP	DEFAULT NOW()	Account creation time
is_active	BOOLEAN	DEFAULT TRUE	Account status flag

Permit

Column	Type	Constraints	Description
permit_id	SERIAL	PRIMARY KEY	Unique permit record
user_id	INTEGER	FK → User, NOT NULL	Permit owner
lot_id	INTEGER	FK → ParkingLot, NOT NULL	Designated lot
permit_type	VARCHAR(30)	NOT NULL	RES/COMMUTER/ EMPLOYEE/ VISITOR
start_date	DATE	NOT NULL	Validity start
end_date	DATE	NOT NULL, >= start	Validity end
cost_usd	DECIMAL(8,2)	CHECK >= 0	Amount paid
vehicle_plate	VARCHAR(15)	NOT NULL	Registered license plate

CrowdsourcedReport

Design note: The SQL table stores only the lightweight, indexable fields needed for aggregation and dashboard statistics (lot, time, status). The full enriched document (device info, media URLs, AI sentiment) is stored exclusively in the MongoDB crowdsourced_reports collection. The shared key is report_id, which MongoDB stores as sql_report_id. This eliminates any duplication of raw report payloads.

Column	Type	Constraints	Description
report_id	SERIAL	PRIMARY KEY	Unique report; mirrored in MongoDB as sql report_id
user_id	INTEGER	FK → User	Submitting user (NULL if anonymous)
lot_id	INTEGER	FK→ ParkingLot, NOT NULL	Reported lot
reported_at	TIMESTAMP	DEFAULT NOW()	Submission time
status	VARCHAR(20)	NOT NULL, CHECK	found_spot/lot_full/hard_to_find
approx_available	INTEGER	CHECK >= 0	User estimated open spots
source_type	VARCHAR(20)	DEFAULT 'user'	user / admin_seed / simulated

CampusEvent

Column	Type	Constraints	Description
event_id	SERIAL	PRIMARY KEY	Unique event
event_name	VARCHAR(150)	NOT NULL	e.g., 'Spring Commencement'
event_date	DATE	NOT NULL	Date of event
start_time	TIME	NOT NULL	Event start time
end_time	TIME		Event end (NULL if open-ended)
location	VARCHAR(100)		Campus venue
attendance_est	INTEGER	CHECK >= 0	Expected attendees
impact_level	VARCHAR(10)	DEFAULT 'medium'	low / medium / high

EventLotImpact (Junction: CampusEvent ↔ ParkingLot)

Column	Type	Constraints	Description
event_id	INTEGER	PK, FK → CampusEvent	Associated event
lot_id	INTEGER	PK, FK → ParkingLot	Affected lot
demand_multiplier	DECIMAL(4,2)	DEFAULT 1.5	Demand boost factor during event

WeatherSnapshot

Column	Type	Constraints	Description
weather_id	SERIAL	PRIMARY KEY	Unique weather record
recorded_at	TIMESTAMP	NOT NULL	Hourly observation time
temperature_f	DECIMAL(5,1)		Temperature in °F
condition	VARCHAR(30)	NOT NULL, CHECK	clear / cloudy / rain / snow / blizzard / fog
precipitation_in	DECIMAL(5,2)	DEFAULT 0	Precipitation in inches
wind_speed_mph	DECIMAL(5,1)		Wind speed in mph
source	VARCHAR(50)		e.g., OpenWeatherMap

ParkingPrediction

Context: factors_summary is typed JSONB (not TEXT) so PostgreSQL can index individual factor keys and support queries like: WHERE factors_summary->>'weather' = 'snow'

Column	Type	Constraints	Description
pred_id	SERIAL	PRIMARY KEY	Unique prediction record
lot_id	INTEGER	FK → ParkingLot	Lot being predicted
weather_id	INTEGER	FK → WeatherSnapshot	Associated weather snapshot
predicted_at	TIMESTAMP	NOT NULL	When prediction was generated
target_time	TIMESTAMP	NOT NULL	Time the prediction applies to
prob_score	DECIMAL(5,4)	CHECK 0.0 1.0	Probability of finding a spot
confidence_level	VARCHAR(10)		low / medium / high
factors_summary	JSONB		Structured factor breakdown (see note above)
model_version	VARCHAR(20)		Algorithm version that produced this score

B.3 SQL Implementation Scripts

B.3.1 Table Creation & Constraints

```
-- HuskyPark Predictor | PostgreSQL DDL
-- Database: huskypark_db
CREATE SCHEMA IF NOT EXISTS huskypark;
SET search_path TO huskypark;

-- 1. ParkingLot
CREATE TABLE ParkingLot (
   lot_id        SERIAL        PRIMARY KEY,
   lot_name      VARCHAR(50)   NOT NULL UNIQUE,
   lot_type      VARCHAR(20)   NOT NULL
                 CHECK (lot_type IN ('resident','commuter','employee','pay')),
   permit_type   VARCHAR(30)   NOT NULL,
   total_spots   INTEGER       NOT NULL CHECK (total_spots > 0),
   is_indoor     BOOLEAN       NOT NULL DEFAULT FALSE,
   latitude      DECIMAL(9,6)  NOT NULL,
   longitude     DECIMAL(9,6)  NOT NULL,
   description   TEXT
);

-- 2. ParkingZone
CREATE TABLE ParkingZone (
   zone_id   SERIAL      PRIMARY KEY,
```

```
    lot_id    INTEGER     NOT NULL REFERENCES ParkingLot(lot_id) ON DELETE CASCADE,
    zone_name VARCHAR(40) NOT NULL,
    capacity  INTEGER     NOT NULL CHECK (capacity >= 0),
    zone_type VARCHAR(20) DEFAULT 'Standard'
            CHECK (zone_type IN ('Standard','Accessible','Reserved','EV'))
);

-- 3. User
CREATE TABLE "User" (
    user_id    SERIAL       PRIMARY KEY,
    full_name  VARCHAR(100) NOT NULL,
    email      VARCHAR(150) NOT NULL UNIQUE,
    user_type  VARCHAR(20)  NOT NULL
            CHECK (user_type IN ('student','faculty','staff','visitor')),
    star_id    VARCHAR(20)  UNIQUE,
    created_at TIMESTAMP    NOT NULL DEFAULT NOW(),
    is_active  BOOLEAN      NOT NULL DEFAULT TRUE
);

-- 4. Permit
CREATE TABLE Permit (
    permit_id     SERIAL        PRIMARY KEY,
    user_id       INTEGER       NOT NULL REFERENCES "User"(user_id),
    lot_id        INTEGER       NOT NULL REFERENCES ParkingLot(lot_id),
    permit_type   VARCHAR(30)   NOT NULL,
    start_date    DATE          NOT NULL,
    end_date      DATE          NOT NULL,
    cost_usd      DECIMAL(8,2)  CHECK (cost_usd >= 0),
    vehicle_plate VARCHAR(15)   NOT NULL,
    issued_at     TIMESTAMP     NOT NULL DEFAULT NOW(),
    CONSTRAINT chk_permit_dates CHECK (end_date >= start_date)
);

-- 5. CrowdsourcedReport
--    Lightweight SQL record for aggregation only.
--    Enriched payload lives in MongoDB (linked by report_id).
CREATE TABLE CrowdsourcedReport (
    report_id       SERIAL      PRIMARY KEY,
    user_id         INTEGER     REFERENCES "User"(user_id) ON DELETE SET NULL,
    lot_id          INTEGER     NOT NULL REFERENCES ParkingLot(lot_id),
    reported_at     TIMESTAMP   NOT NULL DEFAULT NOW(),
    status          VARCHAR(20) NOT NULL
                CHECK (status IN ('found_spot','lot_full','hard_to_find')),
    approx_available INTEGER    CHECK (approx_available >= 0),
    source_type     VARCHAR(20) NOT NULL DEFAULT 'user'
                CHECK (source_type IN ('user','admin_seed','simulated'))
);

-- 6. CampusEvent
CREATE TABLE CampusEvent (
    event_id       SERIAL       PRIMARY KEY,
    event_name     VARCHAR(150) NOT NULL,
    event_date     DATE         NOT NULL,
    start_time     TIME         NOT NULL,
    end_time       TIME,
    location       VARCHAR(100),
    attendance_est INTEGER      CHECK (attendance_est >= 0),
    impact_level   VARCHAR(10)  DEFAULT 'medium'
                CHECK (impact_level IN ('low','medium','high'))
);
```

```
-- 7. EventLotImpact (M:N Junction)
CREATE TABLE EventLotImpact (
   event_id         INTEGER      NOT NULL REFERENCES CampusEvent(event_id)
                          ON DELETE CASCADE,
   lot_id           INTEGER      NOT NULL REFERENCES ParkingLot(lot_id)
                          ON DELETE CASCADE,
   demand_multiplier DECIMAL(4,2) NOT NULL DEFAULT 1.5
                CHECK (demand_multiplier >= 1.0),
   PRIMARY KEY (event_id, lot_id)
);

-- 8. WeatherSnapshot
CREATE TABLE WeatherSnapshot (
   weather_id       SERIAL        PRIMARY KEY,
   recorded_at      TIMESTAMP     NOT NULL,
   temperature_f    DECIMAL(5,1),
   condition        VARCHAR(30)   NOT NULL
                CHECK (condition IN
                 ('clear','cloudy','rain','snow','blizzard','fog')),
   precipitation_in DECIMAL(5,2)  DEFAULT 0 CHECK (precipitation_in >= 0),
   wind_speed_mph   DECIMAL(5,1),
   source           VARCHAR(50)
);

-- 9. ParkingPrediction  (JSONB for factors_summary)
CREATE TABLE ParkingPrediction (
   pred_id          SERIAL        PRIMARY KEY,
   lot_id           INTEGER       NOT NULL REFERENCES ParkingLot(lot_id),
   weather_id       INTEGER       REFERENCES WeatherSnapshot(weather_id),
   predicted_at     TIMESTAMP     NOT NULL DEFAULT NOW(),
   target_time      TIMESTAMP     NOT NULL,
   prob_score       DECIMAL(5,4)  NOT NULL CHECK (prob_score BETWEEN 0 AND 1),
   confidence_level VARCHAR(10)   DEFAULT 'medium'
                CHECK (confidence_level IN ('low','medium','high')),
   factors_summary  JSONB,
   model_version    VARCHAR(20)
);

-- GIN index on JSONB column for factor-level queries
CREATE INDEX idx_prediction_factors
   ON ParkingPrediction USING GIN (factors_summary);
```

B.3.2 Performance Indexes

```
-- 1. Dashboard primary query: latest prediction per lot
CREATE INDEX idx_prediction_lot_time
   ON ParkingPrediction (lot_id, target_time DESC);

-- 2. Crowd report lookups: lot timeline for aggregation
CREATE INDEX idx_report_lot_time
   ON CrowdsourcedReport (lot_id, reported_at DESC);

-- 3. Event lookup by date
CREATE INDEX idx_event_date
   ON CampusEvent (event_date);
```

```
-- 4. Weather correlation by timestamp
CREATE INDEX idx_weather_time
   ON WeatherSnapshot (recorded_at DESC);

-- 5. Permit lookup per user
CREATE INDEX idx_permit_user
   ON Permit (user_id, end_date);

-- 6. Lot type + permit type for filtered dashboard views
CREATE INDEX idx_lot_type
   ON ParkingLot (lot_type, permit_type);
```

B.3.3 Views

```
-- View 1: Latest prediction per lot (uses DISTINCT ON for correctness)
CREATE OR REPLACE VIEW vw_latest_predictions AS
SELECT DISTINCT ON (p.lot_id)
   p.pred_id,
   p.lot_id,
   l.lot_name,
   l.lot_type,
   l.permit_type,
   p.prob_score,
   p.confidence_level,
   p.target_time,
   p.predicted_at,
   p.factors_summary,
   p.model_version
FROM ParkingPrediction p
JOIN ParkingLot l ON l.lot_id = p.lot_id
ORDER BY p.lot_id, p.target_time DESC;

-- View 2: Hourly report counts per lot (last 24 hours)
CREATE OR REPLACE VIEW vw_hourly_report_counts AS
SELECT
   lot_id,
   date_trunc('hour', reported_at)                    AS report_hour,
   COUNT(*)                                           AS total_reports,
   SUM(CASE WHEN status='found_spot'   THEN 1 ELSE 0 END) AS found,
   SUM(CASE WHEN status='lot_full'     THEN 1 ELSE 0 END) AS full,
   SUM(CASE WHEN status='hard_to_find' THEN 1 ELSE 0 END) AS hard
FROM CrowdsourcedReport
WHERE reported_at >= NOW() - INTERVAL '24 hours'
  AND source_type = 'user'
GROUP BY lot_id, report_hour
ORDER BY lot_id, report_hour;

-- View 3: Active events today with affected lots
CREATE OR REPLACE VIEW vw_active_events AS
SELECT
   e.event_id,
   e.event_name,
   e.start_time,
   e.end_time,
   e.impact_level,
   e.attendance_est,
   STRING_AGG(l.lot_name, ', ' ORDER BY l.lot_name) AS affected_lots
FROM CampusEvent e
JOIN EventLotImpact eli ON eli.event_id = e.event_id
JOIN ParkingLot     l   ON l.lot_id     = eli.lot_id
```

```
WHERE e.event_date = CURRENT_DATE
GROUP BY e.event_id, e.event_name, e.start_time,
        e.end_time, e.impact_level, e.attendance_est;
```

C. NoSQL Database Design

The NoSQL layer uses MongoDB. The key architectural decision is that MongoDB stores exclusively data that cannot be efficiently modeled in PostgreSQL: high velocity enriched reports, AI session logs with variable structure, and aggregated analytics snapshots. No data is duplicated between the two systems.

C.1 SQL vs MongoDB: Clear Data Boundary

What it is	Where it lives	Reason
Lot/zone definitions, user accounts, permits	PostgreSQL only	Structured, relational, requires strong consistency
Rawreport record (lot_id, status, reported_at)	PostgreSQL only	Lightweight row needed for aggregation queries and FK integrity; shared report_id links to MongoDB for the full payload
Enriched report payload (device info, photos,GPS accuracy, AI sentiment, optional media URLs)	MongoDB only	Variable fields per submission; grows as the app evolves; photo URL list is variable-length, bad fit for SQL
Prediction JSONB factors (inline on ParkingPrediction row)	PostgreSQL only (JSONB column)	Small structured object; GIN indexed; stays close to its prediction row
AI recommendation sessions (full dialogue, ranked lots, LLM metadata)	MongoDB only	Session structure varies; includes nested arrays of recommendations
Notification logs	MongoDB only	Payload shape differs per notification type
Hourly lot analytics (trends, model output history)	MongoDB only	Pre-aggregated documents; evolves as prediction model changes

C.2 Why MongoDB Is Appropriate

- **Flexible crowdsourced data:** Each user report can include optional photos, device metadata, and varying weather detail. A rigid SQL row would require NULLs for most fields; a document naturally omits absent keys.
- **High write throughput:** During peak hours (8–9 AM, 12–1 PM), hundreds of spot-status reports may arrive per minute. MongoDB's write performance and horizontal sharding handle this load better than a single PostgreSQL write path.
- **AI session logs:** AI generated recommendation sessions vary widely in structure. Document storage accommodates this variability.
- **Evolving schema:** As new prediction factors are identified, the document model allows new fields to be added without migrations.

C.3 Collections & Sample Documents

Collection 1: crowdsourced_reports (enriched payload only)

SQL stores the lightweight record. MongoDB stores the full document, linked by sql_report_id. The application writes to SQL first, then MongoDB. If the MongoDB write fails, the core record is preserved in SQL and the MongoDB write is retried asynchronously.

```
{
 "_id":          "ObjectId(64f1a2b3c4d5e6f7a8b9c0d1)",
 "sql_report_id": 1042,
 "lot_id":       7,
 "lot_name":     "Lot C",
 "user_id":      "usr_1042",
 "reported_at":  "2026-03-16T08:47:12Z",
 "status":       "found_spot",
 "approx_available": 12,
 "source_type":  "user",
 "weather": {
  "condition":       "snow",
  "temperature_f":   22.4,
  "precipitation_in": 0.3
 },
 "device_info": {
  "platform":          "iOS",
  "app_version":       "1.2.0",
  "location_accuracy_m": 5
 },
 "image_urls": [
  "https://cdn.huskypark.scsu.edu/reports/64f1a2b3_1.jpg"
 ],
 "sentiment_score": 0.72,
 "tags":   ["snowy", "found quickly"],
 "verified": false
}
```

Collection 2: ai_recommendation_sessions

```
{
 "_id":       "ObjectId(64f2c3d4e5f6a7b8c9d0e1f2)",
 "user_id":   "usr_1042",
```

```
 "user_type":  "student",
 "permit_type":"COMMUTER",
 "created_at": "2026-03-16T08:48:00Z",
 "query": {
  "text":       "Where can I park near Atwood at 9 AM?",
  "target_time": "2026-03-16T09:00:00Z",
  "target_area": "Atwood Memorial Center"
 },
 "context_snapshot": {
  "weather_condition": "snow",
  "active_events":     ["Admissions Open House"],
  "campus_status":     "normal"
 },
 "recommendations": [
   {
    "rank":       1,
    "lot_id":     7,
    "lot_name":   "Lot C",
    "prob_score": 0.78,
    "distance_m": 120,
    "rationale":  "Best availability for commuter permits near Atwood."
   },
   {
    "rank":       2,
    "lot_id":     12,
    "lot_name":   "Lot M",
    "prob_score": 0.61,
    "distance_m": 210,
    "rationale":  "Good fallback; slightly farther but less crowded."
   }
 ],
 "ai_response_text": "Based on current conditions, Lot C is your best bet...",
 "model_used":       "gpt-4o-mini",
 "latency_ms":       412,
 "user_feedback":    null
}
```


Collection 3: lot_hourly_analytics

Pre aggregated analytics document per lot per hour. Reduces real time computation on the dashboard and stores evolving model output history as the algorithm improves.


```
{
 "_id":        "lot_C_2026_03_16_08",
 "lot_id":      7,
 "lot_code":    "C",
 "date":       "2026-03-16",
 "hour":        8,
 "day_of_week": "Monday",
 "permit_type": "COMMUTER",
 "historical_patterns": {
  "avg_reports_this_hour":   24,
  "avg_prob_available_pct":  42.8,
  "common_status":           "hard_to_find"
 },
 "context_at_snapshot": {
  "weather": {
   "condition":       "snow",
   "temperature_f":    22,
```

```
    "precipitation_in": 0.3
  },
  "events": [
    { "event_name": "Admissions Open House", "impact_level": "medium" }
  ]
},
"trend_summary": {
  "last_7_days_avg_pct":  48.1,
  "last_30_days_avg_pct": 51.7
},
"model_output": {
  "prob_score":       0.375,
  "predicted_status": "limited",
  "model_version":    "v1.2",
  "generated_at":     "2026-03-16T08:00:00Z"
}
}
```


C.4 MongoDB Index Strategy

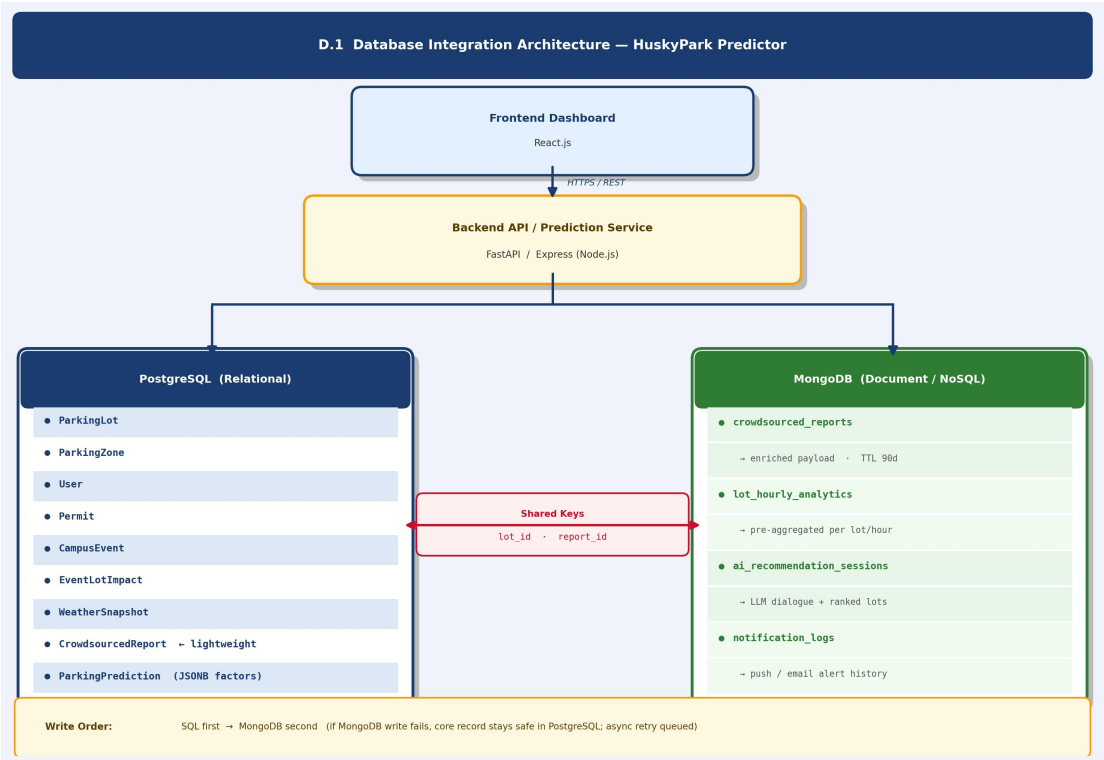
```
// crowdsourced_reports
db.crowdsourced_reports.createIndex({ lot_id: 1, reported_at: -1 });
db.crowdsourced_reports.createIndex({ sql_report_id: 1 }, { unique: true });
db.crowdsourced_reports.createIndex(
    { reported_at: 1 },
    { expireAfterSeconds: 7776000 }  // TTL: auto-delete after 90 days
);

// lot_hourly_analytics
db.lot_hourly_analytics.createIndex({ lot_id: 1, date: 1, hour: 1 });

// ai_recommendation_sessions
db.ai_recommendation_sessions.createIndex({ user_id: 1, created_at: -1 });
```

D. Database Integration Plan

D.1 Architecture Overview



D.2 Data Responsibility Separation

The relational database stores all parking lot definitions, user accounts, permits, campus events, weather snapshots, and core prediction records. These data elements are structured, strongly related, and benefit from constraints, consistency, and efficient joins. MongoDB stores enriched crowdsourced report payloads, AI recommendation sessions, notification logs, and pre-aggregated hourly analytics, data that is flexible in structure, high in write volume, or evolves over time.

D.3 Application data Flow

Flow 1: User Submits a Spot Report

Step 1: React Dashboard The user selects a lot on the dashboard and submits a spot status (found spot, lot full, or hard to find).

Step 2: POST /api/reports The React frontend sends a POST request to the backend API with the lot ID, status, and any optional details such as estimated available spots.

Step 3: Validate lot_id in PostgreSQL The backend queries the ParkingLot table in PostgreSQL to confirm the submitted lot_id exists and is active before accepting the report.

Step 4: Insert row into CrowdsourcedReport (SQL) A lightweight record is written to the CrowdsourcedReport table in PostgreSQL. This row stores only the fields needed for aggregation: lot_id, user_id, reported_at, status, approx_available, and source_type.

Step 5: Write enriched document to MongoDB The backend writes the full enriched document to the MongoDB crowdsourced_reports collection. This document includes the sql_report_id from Step 4 as the linking key, along with additional fields such as device info, weather at time of report, image URLs, and AI sentiment score.

Step 6: Trigger async prediction recalculation A background task is queued to recalculate the probability score for the affected lot and write an updated record to the ParkingPrediction table in PostgreSQL.

Flow 2: Dashboard Loads

Step 1: React Requests Dashboard The user opens the HuskyPark dashboard. The React frontend sends a request to the backend for all current lot data.

Step 2: GET /api/dashboard The backend API receives the request and prepares to query both databases simultaneously.

Step 3: Query vw_latest_predictions (PostgreSQL) The backend queries the vw_latest_predictions view in PostgreSQL. This view uses DISTINCT ON to return exactly one row per lot — the most recent probability score, confidence level, and JSONB factors summary.

Step 4: Query lot_hourly_analytics (MongoDB) The backend queries the MongoDB lot_hourly_analytics collection for the current hour's pre-aggregated analytics document for each lot, including historical trend summaries and model output history.

Step 5: Merge on lot_id in-memory The backend merges the PostgreSQL result set and the MongoDB result set in application memory, using lot_id as the common key. Each lot entry in the final response contains both the live prediction score and the historical analytics context.

Step 6: Return combined JSON to React The merged payload is serialized to JSON and returned to the React dashboard, which renders the probability indicators, trend summaries, and lot status map.

Flow 3: AI Recommendation Request

Step 1: User Requests Recommendation The user types a natural language query into the dashboard, such as "Where can I park near Atwood at 9 AM?" and submits it.

Step 2: POST /api/recommend The React frontend sends the query text, target time, and target area to the backend recommendation endpoint.

Step 3: Fetch lots and WeatherSnapshot (PostgreSQL) The backend queries PostgreSQL for the list of lots matching the user's permit type, their current probability scores from vw_latest_predictions, and the most recent WeatherSnapshot record to use as context.

Step 4: Call LLM with context The backend constructs a structured prompt containing the lot options, probability scores, weather conditions, and any active campus events. This prompt is sent to the language model (e.g., GPT-4o-mini) to generate a ranked recommendation with natural language rationale.

Step 5: Store session in MongoDB ai_recommendation_sessions The full session document is written to MongoDB. This includes the original query, the context snapshot, the ranked recommendations with scores and rationale, the AI response text, the model used, and the response latency in milliseconds.

Step 6: Return ranked lots to React The backend returns the ranked lot recommendations and the AI-generated explanation text to the React frontend for display on the dashboard.

Flow 4: Failure Handling (MongoDB Write Fails)

Step 1: SQL write succeeds The core record, whether a CrowdsourcedReport row or a ParkingPrediction row is successfully written to PostgreSQL. The critical operational data is now safe.

Step 2: MongoDB write fails The subsequent write to MongoDB fails due to a network timeout, connection issue, or service interruption.

Step 3: Core record preserved in PostgreSQL Because the write order is SQL first and MongoDB second, no data is lost. The PostgreSQL record stands on its own and the dashboard can still serve prediction data from it.

Step 4: Failed write queued for async retry The backend logs the failure and places the MongoDB write job onto an async retry queue (e.g., Celery or a Redis queue). The retry will attempt the MongoDB write again after a short delay without blocking the user's request.

Step 5: Dashboard degrades gracefully The frontend continues to function using the PostgreSQL only data. Prediction scores and lot availability remain visible. Only the enriched analytics layer (trends, AI session history) is temporarily unavailable until the MongoDB sync recovers.

Step 6: MongoDB sync recovers and retries Once the MongoDB connection is restored, the queued write is processed and the enriched document is written. The system returns to full operation with no data loss.

D.4 Data Consistency Strategy

- SQL is the source of truth for all core operational records.
- MongoDB stores derived or supplemental data only.
- The shared link between systems is lot_id (integer) for analytics, and report_id/sql_report_id for crowd reports.
- The backend always writes to PostgreSQL first; MongoDB is written second.
- A nightly reconciliation job aggregates MongoDB report counts per lot per day and writes the summary to a PostgreSQL DailySummary table for long-term analytics.
- MongoDB's TTL index (90 days) on crowdsourced_reports prevents unbounded growth without requiring synchronization with PostgreSQL.

E. Performance & Optimization

E.1 Indexing Strategy

Index	DB	Column(s)	Justification
idx_prediction_lot_time	PostgreSQL	(lot_id, target_time DESC)	Supports DISTINCT ON query in vw_latest_predictions
idx_prediction_factors	PostgreSQL	factors_summary (GIN)	Enables JSONB key/valuefiltering on prediction factors
idx_report_lot_time	PostgreSQL	(lot_id, reported_at DESC)	2 hour crowd report aggregation per lot
idx_event_date	PostgreSQL	event_date	O(log n) active event lookup for prediction engine
idx_weather_time	PostgreSQL	recorded_at DESC	Nearest weather correlation on prediction generation
{lot_id, reported_at}	MongoDB	crowdsourced_reports	Primary read path: recent reports per lot
TTL on reported_at	MongoDB	crowdsourced_reports	Auto delete documents after 90 days; zero maintenance
{lot_id, date, hour}	MongoDB	lot_hourly_analytics	Dashboard retrieval of current hour analytics snapshot

E.2 Query Optimization Considerations

- Use vw_latest_predictions with DISTINCT ON to return exactly one row per lot, avoids returning thousands of historical prediction rows to the dashboard.
- The source_type = 'user' filter in vw_hourly_report_counts excludes simulated seed data from live crowd views without deleting it.
- Store only derived analytics in MongoDB; avoid expensive repeated aggregation at request time by pre computing the lot_hourly_analytics documents every 15 minutes via a background scheduler.
- JSONB GIN index on factors_summary enables future analytics queries such as "how often does snow reduce prob_score below 0.3 in Lot C?" without schema changes.

E.3 Scalability Discussion

- Read replicas (PostgreSQL): Dashboard prediction reads routed to read replicas; writes go to primary. Achievable with asyncpg connection string routing.
- MongoDB sharding on lot_id: As the platform expands to multiple campuses, shard crowdsourced_reports and lot_hourly_analytics on lot_id for horizontal write scaling.
- Redis cache: Cache vw_latest_predictions result with a 60 second TTL to absorb the burst of simultaneous dashboard loads at 8 AM when hundreds of commuters check lots at once.
- Pre-computation: A background scheduler (APScheduler / Celery) recomputes predictions every 15 minutes and writes to ParkingPrediction, keeping dashboard reads instant rather than triggering live ML inference on every page load.

Appendix: Realistic SCSU Seed Data

All seed lots use real SCSU lot names, accurate lot types, and GPS coordinates. Seed crowd reports carry source_type = 'simulated' so they are excluded from live dashboard views.

```
-- Seed: ParkingLot (real SCSU lots)
INSERT INTO huskypark.ParkingLot
  (lot_name, lot_type, permit_type, total_spots, is_indoor, latitude, longitude)
VALUES
  -- COMMUTER / STUDENT LOTS
  ('Lot A',        'commuter', 'COMMUTER-A',  120, FALSE, 45.5603, -94.1512),
  ('Lot C',        'commuter', 'COMMUTER-C',  185, FALSE, 45.5598, -94.1505),
  ('Lot E',        'commuter', 'COMMUTER-E',   95, FALSE, 45.5595, -94.1520),
  ('Lot K',        'commuter', 'COMMUTER-K',  210, FALSE, 45.5609, -94.1495),
  ('Lot M',        'commuter', 'COMMUTER-M',  165, FALSE, 45.5601, -94.1530),
  ('Lot V',        'commuter', 'COMMUTER-V',   88, FALSE, 45.5592, -94.1540),
  ('Stateview Lot', 'commuter', 'COMMUTER-SV', 140, FALSE, 45.5578, -94.1558),
  -- RESIDENT LOTS
```

```
  ('Lot N',         'resident', 'RESIDENT-N',   80, FALSE, 45.5612, -94.1488),
  ('RES Lot 1',     'resident', 'RESIDENT-R1',  75, FALSE, 45.5618, -94.1480),
  ('RES Lot 2',     'resident', 'RESIDENT-R2',  70, FALSE, 45.5621, -94.1474),
  -- EMPLOYEE LOTS
  ('Lot AA',        'employee', 'EMPLOYEE-AA', 110, FALSE, 45.5590, -94.1545),
  ('Lot B',         'employee', 'EMPLOYEE-B',  145, FALSE, 45.5585, -94.1552),
  ('Lot D',         'employee', 'EMPLOYEE-D',   90, FALSE, 45.5580, -94.1560),
  -- PAY LOTS
  ('4th Ave Ramp',  'pay',      'PAY-RAMP',    420, TRUE,  45.5620, -94.1465),
  ('Husky Lot',     'pay',      'PAY-HUSKY',   140, FALSE, 45.5630, -94.1460),
  ('Eastman Lot',   'pay',      'PAY-EASTMAN', 100, FALSE, 45.5625, -94.1470),
  ('ISELF Lot',     'pay',      'PAY-ISELF',    85, FALSE, 45.5635, -94.1455),
  ('Miller Lot',    'pay',      'PAY-MILLER',   60, FALSE, 45.5640, -94.1450),
  ('South Lot',     'pay',      'PAY-SOUTH',   110, FALSE, 45.5575, -94.1575);

-- Seed: CampusEvent
INSERT INTO huskypark.CampusEvent
  (event_name, event_date, start_time, end_time, location, attendance_est, impact_level)
VALUES
  ('Spring Commencement',      '2026-05-02', '10:00', '13:00',
   'Herb Brooks National Hockey Center', 2500, 'high'),
  ('Admissions Open House',    '2026-04-11', '09:00', '14:00',
   'Atwood Memorial Center',             800,  'medium'),
  ('Homecoming Football Game', '2026-10-03', '14:00', '17:00',
   'Husky Athletic Complex',            3000, 'high'),
  ('Career Fair',              '2026-03-18', '09:00', '15:00',
   'Atwood Memorial Center',             600,  'medium'),
  ('Graduate Orientation',     '2026-08-25', '08:00', '12:00',
   'Brown Hall',                         400,  'low');

-- Seed: EventLotImpact
INSERT INTO huskypark.EventLotImpact (event_id, lot_id, demand_multiplier)
VALUES
  (1, 14, 2.5), (1, 15, 2.0), (1, 16, 1.8), (1, 17, 1.5);

-- Seed: Simulated CrowdsourcedReports
INSERT INTO huskypark.CrowdsourcedReport
  (lot_id, reported_at, status, approx_available, source_type)
VALUES
  (2, NOW() - INTERVAL '1 hour',   'found_spot',   18, 'simulated'),
  (2, NOW() - INTERVAL '2 hours',  'hard_to_find',  4, 'simulated'),
  (5, NOW() - INTERVAL '30 mins',  'lot_full',      0, 'simulated'),
  (1, NOW() - INTERVAL '45 mins',  'found_spot',   32, 'simulated'),
  (8, NOW() - INTERVAL '3 hours',  'found_spot',   15, 'simulated');

-- Seed: WeatherSnapshot
INSERT INTO huskypark.WeatherSnapshot
  (recorded_at, temperature_f, condition, precipitation_in, wind_speed_mph, source)
VALUES
  (NOW() - INTERVAL '1 hour',  22.4, 'snow',   0.3, 12.5, 'OpenWeatherMap'),
  (NOW() - INTERVAL '2 hours', 24.1, 'snow',   0.2, 10.0, 'OpenWeatherMap'),
  (NOW() - INTERVAL '3 hours', 27.0, 'cloudy', 0.0,  8.3, 'OpenWeatherMap'),
  (NOW() - INTERVAL '4 hours', 28.5, 'cloudy', 0.0,  7.1, 'OpenWeatherMap');

-- Sample: ParkingPrediction with JSONB factors_summary
INSERT INTO huskypark.ParkingPrediction
  (lot_id, weather_id, target_time, prob_score, confidence_level,
   factors_summary, model_version)
VALUES
  (2, 1, NOW() + INTERVAL '1 hour', 0.3720, 'medium',
```

```
'{"weather":"snow","precipitation":"0.3in","hour_of_day":9,
  "day_of_week":"Monday","active_events":["Admissions Open House"],
  "recent_reports":{"found":2,"full":3,"hard":5}}',
'v1.2');
```

Summary

The relational database was selected for the core operational data of HuskyPark because parking lots, permit categories, user accounts, reports, events, and prediction records have clear relationships and require strong consistency, constraints, and efficient joins. A NoSQL document database was selected to complement the relational system by storing flexible, high frequency enriched reports, AI recommendation sessions, and hourly analytics outputs which may change in structure over time. This separation provides data integrity where needed while improving scalability and flexibility for semi structured analytical data. The JSONB typing of factors_summary bridges both worlds, keeping structured factor data close to its prediction row in PostgreSQL while remaining queryable without schema changes.